detkov / **Convolution-From-Scratch** Public

Implementation of the generalized 2D convolution with dilation from scratch in Python and NumPy

MIT license

49 stars 6 forks Branches Tags Activity

Star Notifications

Code Issues 1 Pull requests Actions Security Insights

main 1 Branch 0 Tags Go to file Go to file Code

detkov Little fixes 9fb976f · 4 years ago

files	Little fixes	4 years ago
.gitignore	Added Docstring, something in README and refactore...	4 years ago
LICENSE.md	Initial commit	4 years ago
README.md	Little fixes	4 years ago
convolution.py	Little fixes	4 years ago
plots.py	Little fixes	4 years ago
tests.py	Added Docstring, something in README and refactore...	4 years ago

Convolution from scratch

Motivation on repository

I tried to find the algorithm of convolution with dilation, implemented from scratch on a pure python, but could not find anything. There are a lot of self-written CNNs on the Internet and on the GitHub and so on, a lot of tutorials and explanations on convolutions, but there is a lack of a very important thing: proper implementation of a generalized 2D convolution for a kernel of any form with adjustable on both axes parameters, such as stride, padding, and most importantly, dilation. The last one cannot be found literally anywhere! This is why this repository and this picture above appeared.

Who needs this?

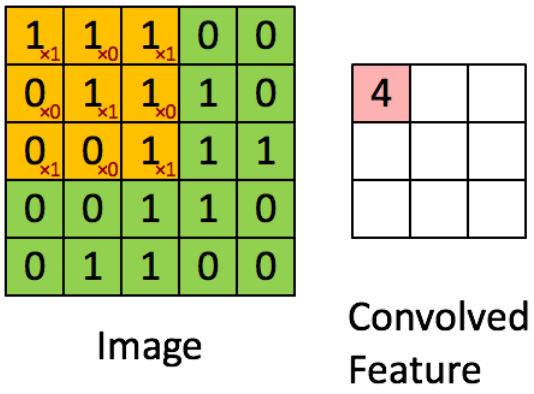
If you've ever wanted to understand how this seemingly simple algorithm can be really implemented in code, this repository is for you. As it turns out, it's not so easy to tie all the parameters together in code to make it general, clear and obvious (and optimal in terms of computations).
Feel free to use it as you wish.

Contents

Explanation

Idea in the nutshell

In 2D convolution we move some small matrix called *Kernel* over 2D *Image* (some matrix) and multiply it element-wise over each sub-matrix, then sum elements of the obtained sub-matrix into a single pixel of so-called *Feature map*. We move it from the left to the right and from the top to the bottom. At the end of convolution we usually cover the whole *Image* surface, but that is not guaranteed with more complex parameters.
This GIF (source) below perfectly presents the essence of the 2D convolution: green matrix is the *Image*, yellow is the *Kernel* and ~~red~~ coral is the *Feature map*:



Let's clarify it and give a definition to every term used:

- **Image** or input data is some matrix;
- **Kernel** is a small matrix that we multiply with sub-matrices of an Image;
  - **Stride** is the size of the step of the slide. For example, when the stride equals 1, we move on 1 pixel on every step, when 2, then we move on 2 pixels and so on. This picture can help you figure it out;
  - **Padding** is just the border of the *Image* that allows us to keep the size of initial *Image* and *Feature map* the same. In the GIF above we see that the shape of *Image* is 5x5 but the *Feature map* is 3x3. The reason is that when we use *Kernel*, we can't put its center in the corner, because if we do, there is a lack of pixels to multiply on. So if we want to keep shape, we use padding and add some zero border of the image. This GIF can help you figure it out;
  - **Dilation** is just the gap between kernel cells. So, the regular dilation is 1 and each cell is not distanced from its neighbor, but when we set the value as 2, there are no cells in the 1-cell neighborhood — now they are distanced from each other. This picture can help you figure it out.
- **Feature map** or output data is the matrix obtained by all the calculations discussed earlier.

This is it — that easy.

Usage

Example with your matrix and kernel

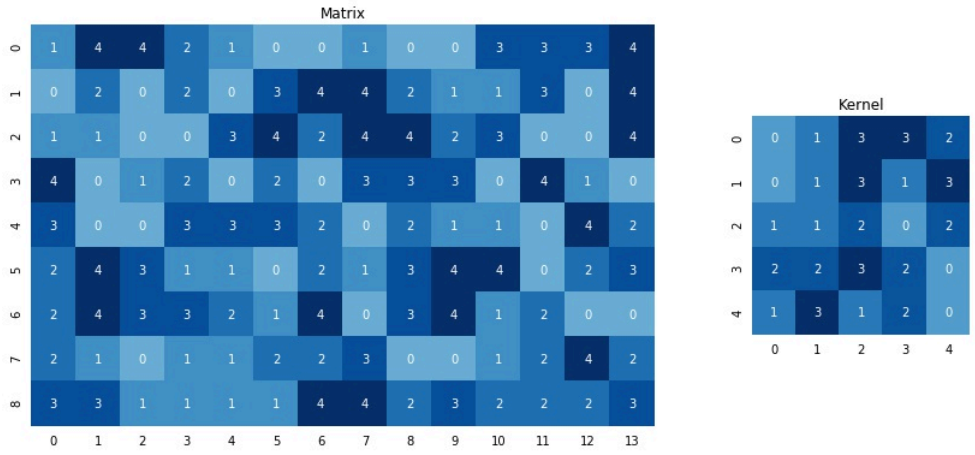
Say, you have a matrix like this one:

```
matrix = np.array([[1, 4, 4, 2, 1, 0, 0, 1, 0, 0, 3, 3, 3, 4],
                   [0, 2, 0, 2, 0, 3, 4, 4, 2, 1, 1, 3, 0, 4],
                   [1, 1, 0, 0, 3, 4, 2, 4, 4, 2, 3, 0, 0, 4],
                   [4, 0, 1, 2, 0, 2, 0, 3, 3, 3, 0, 4, 1, 0],
                   [3, 0, 0, 3, 3, 3, 2, 0, 2, 1, 1, 0, 4, 2],
                   [2, 4, 3, 1, 1, 0, 2, 1, 3, 4, 4, 0, 2, 3],
                   [2, 4, 3, 3, 2, 1, 4, 0, 3, 4, 1, 2, 0, 0],
                   [2, 1, 0, 1, 1, 2, 2, 3, 0, 0, 1, 2, 4, 2],
                   [3, 3, 1, 1, 1, 1, 4, 4, 2, 3, 2, 2, 2, 3]])
```

And a kernel like this one:

```
kernel = np.array([[0, 1, 3, 3, 2],
                   [0, 1, 3, 1, 3],
                   [1, 1, 2, 0, 2],
```

```
        [2, 2, 3, 2, 0],
        [1, 3, 1, 2, 0]])
```

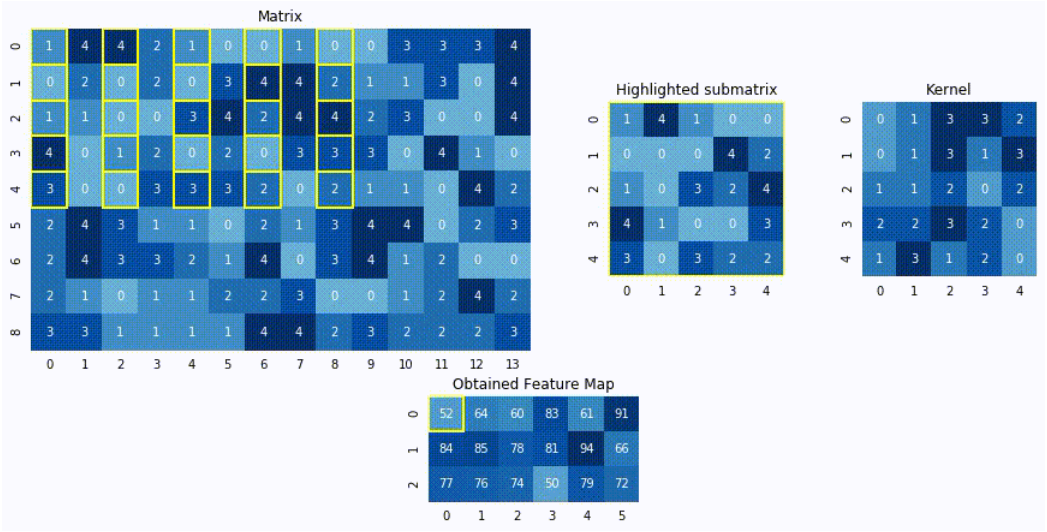


Then, say, you want to apply convolution with `stride = (2, 1)` and `dilation = (1, 2)`. All you need to do is just simply pass it as parameters in `conv2d` function:

```
from convolution import conv2d

feature_map = conv2d(matrix, kernel, stride=(2, 1), dilation=(1, 2), padding=(0, 0))
```

And get the following result:



Example with your image and filter

For example, if you want to blur your image, you can use "Gaussian blur" and take the corresponding kernel, while some others can be found here.

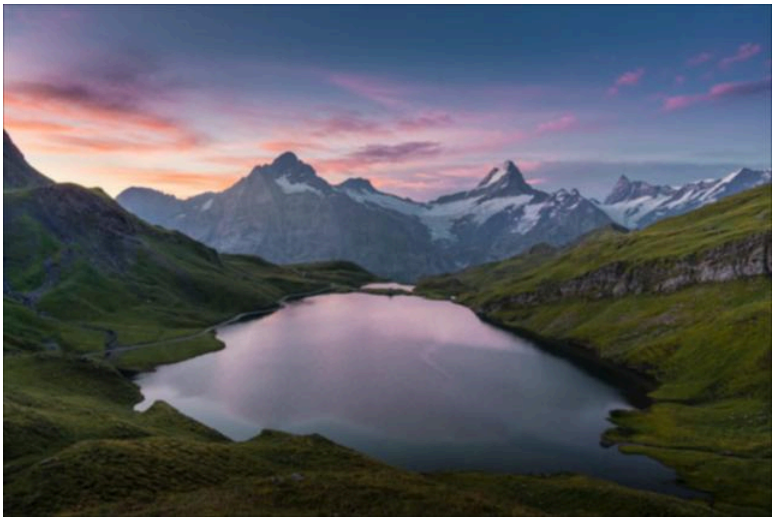
```
import imageio
import matplotlib.pyplot as plt
import numpy as np


gaussian_blur = np.array([
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1]
]) / 16


image = imageio.imread('files/pic.jpg')
plt.imshow(image)
```

Then you just need to use `apply_filter_to_image` function from `convolution.py` module. I'm going to make this picture blurry:

```
filtered_image = apply_filter_to_image(image, gaussian_blur)
plt.imshow(filtered_image)
```



Tadaa, it's blurred!

> P.S. This photo is taken near the alpine lake Bachalpsee in Switzerland (credits).

Running tests

```
python -m unittest tests.py
```

Citation

If you used this repository in your work, consider citing:

```
@misc{Convolution from scratch,
```

Languages

- Python 100.0%